

UNIVERSIDAD NACIONAL DE INGENIERÍA

Intérprete de Lenguaje de Scripting: **MINIPYTHON**

Facultad de Ingeniería Industrial y de Sistemas (FIIS)

Integrantes:

- Arestigue Mamani Erickson
- Jauregui Peña Bruno
- Lavado Quispe Alessandro

Docente / Curso:

Herencia Guerra Juan Luis
Lenguaje de programación imperativo

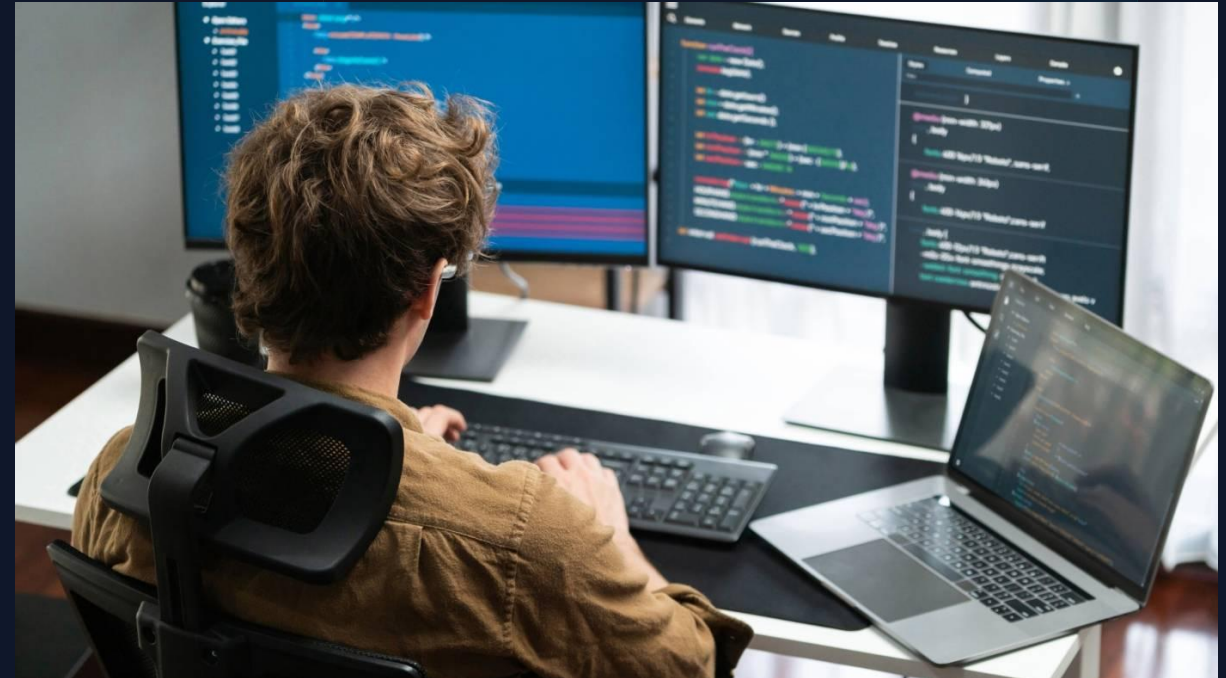
Introducción al Proyecto

Implementación de un intérprete para un mini lenguaje denominado **MINIPYTHON**, diseñado para comprender los fundamentos de la computación.

✓ Inspirado en la sintaxis de Python.

🔗 Arquitectura basada en Lexer y Parser.

⚙️ Evaluación mediante Árbol de Sintaxis Abstracta (AST).



| Objetivos Estratégicos



General

Desarrollar un intérprete funcional utilizando estructuras fundamentales de compiladores para un lenguaje imperativo.



Específicos

Diseñar la gramática formal e implementar el sistema de tokens y el parser recursivo descendente.



Funcionales

Evaluar expresiones aritméticas complejas, manejar variables y asignaciones de datos dinámicos.

| Capacidades del Lenguaje

A Variables: Declaración y uso de identificadores (Ej: `x = 10`).

± Aritmética: Soporte para suma, resta, multiplicación y división.

= Asignación: Almacenamiento de valores mediante el operador de igualdad.

>_ Salida: Instrucción `print()` para visualizar resultados en consola.

Definición de Tokens

Token	Significado	Ejemplo
NUMBER	Valor numérico	5
IDENTIFIER	Nombre de variable	x
PLUS / MINUS	Operadores suma/resta	+, -
MUL / DIV	Operadores mul/div	*, /
PRINT	Palabra reservada	print

| Gramática Formal (BNF)

```
programa ::= lista_de_ordenes  
orden ::= IDENT "=" expr | "print" "(" expr ")"  
expr ::= termino { (+ | -) termino }
```

Precedencia

El parser maneja prioridades:

- ^ Multiplicación y División.
- ✓ Suma y Resta.

Estructura AST

El Árbol de Sintaxis Abstracta (AST) representa la lógica del programa sin detalles sintácticos innecesarios.

NumberNode

AssignNode

BinaryOpNode

PrintNode



Evaluación del Árbol

Asignación

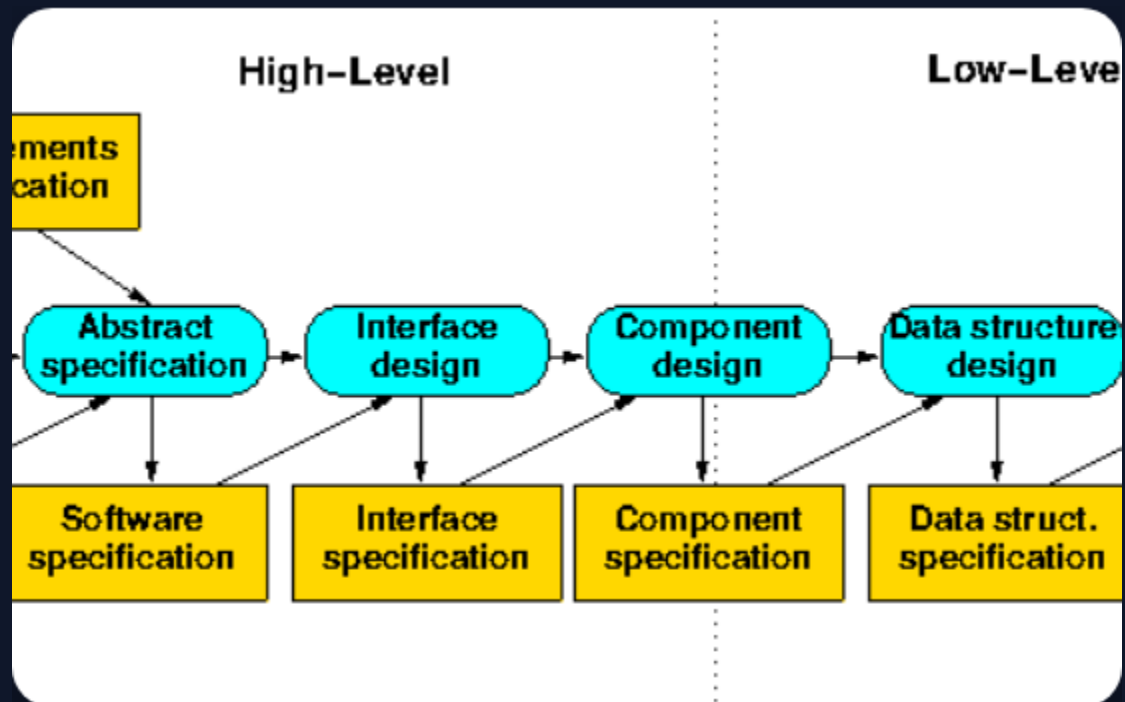
Evalúa la expresión y guarda el resultado en una tabla de símbolos vinculada al identificador.

Operación Binaria

Evalúa ambos operandos recursivamente y aplica el operador matemático correspondiente.

El intérprete recorre el AST de forma recursiva (Tree Walking) para ejecutar cada nodo.

Ejemplo de Flujo Completo



Entrada: $x = 5 + 3$

1. Lexer crea tokens: **ID**, **=**, **NUM**, **+**, **NUM**
2. Parser construye **AssignNode** con hijo **BinaryOp**.
3. Evaluador calcula $5+3 = 8$.
4. Se almacena **x : 8** en memoria.

| Arquitectura del Sistema



Front-end

`lexer.c` y `parser.c` procesan el código fuente y crean el AST.



Core

`ast.c` define los nodos que componen la estructura del programa.



Back-end

`eval.c` y `main.c` ejecutan la lógica y gestionan la consola.

| Estado del Proyecto

Completado

- ✓ Diseño de Gramática BNF.
- ✓ Definición de Tokens.
- ✓ Diseño General del Parser.

Próximos Pasos

- ⋮ Manejo de errores avanzado.
- ⋮ Condicionales if y bucles.
- ⋮ Funciones definidas por usuario.

¿Preguntas?

Gracias por su atención

Proyecto Intérprete MINIPYTHON

UNI - FIIS 2026
